Our sortTargets method takes the list of the elements that are sortElementTargets and, well, sorts them, using the value of their data-sort-value attribute for comparison. Once it has a sorted list, it reappends them in order back to the parent, which has the effect of having all the children display in order. We also have a guard clause where nothing happens if the targets are already sorted, which also should help prevent the MutationObserver from looping endlessly since the changes to the element from appending objects also qualify as mutations.

This SortController is pretty generic, and we could use it almost anywhere to keep a list of items sorted without any further JavaScript.

This works. What's nice is that the timing is such that the sorting happens before the display so that our incoming element only appears to enter the DOM once in the correct location (rather than appearing at the bottom and then moving to the correct location), and the animation still fires, giving the new object the appearance of entering from the right in the correctly sorted location.

There's one little problem—as currently written, the animation happens too often. When a new favorite is added, all the favorites with animate_in as true reanimate and slide in because the animation class is still in the element's class list and the element is getting removed and reinserted, triggering the animation again. What we want is to have the animate__slideInRight class removed after the element slides in once.

While we could do this with another custom controller or by modifying the sort controller, we can actually also do it without writing any new JavaScript with our existing CssController, which also manages whether a CSS class is in or out of an element based on its status. Here's what I did:

chapter_10/03/app/views/favorites/_favorite.html.erb

```
<%- animate_in ||= false %>
<article
  class="my-6 animate__animated"
  id="<%= dom_id(favorite) %>"
  data-animate-out="animate__slideOutRight"
  data-sort-target="sortElement"
```